

Complete Guide: Controlling Fulling iSVD78C-48V-230-A14-002 Motor from Linux

Table of Contents

1. [Hardware Overview](#)
 2. [Hardware Requirements](#)
 3. [Linux System Setup](#)
 4. [CAN Protocol Overview](#)
 5. [Control Modes](#)
 6. [Command-Line Control](#)
 7. [C Programming Examples](#)
 8. [Python Programming Examples](#)
 9. [Troubleshooting](#)
 10. [Reference Tables](#)
-

Hardware Overview

Motor Specifications

- **Model:** iSVD78C-48V-230-A14-002
- **Rated Voltage:** 48 VDC
- **Operating Voltage Range:** 24V-60 VDC
- **Rated Torque:** 6 N.m
- **Peak Torque:** 17 N.m
- **Rated Speed:** 360 rpm $\pm 10\%$
- **No-load Speed:** 410 rpm $\pm 10\%$
- **Reduction Ratio:** 7.75:1
- **Encoder Resolution:** 14-bit absolute encoder
- **CAN Baud Rate:** 1 Mbps (fixed)
- **Communication:** CAN 2.0 with 29-bit extended frames
- **Drive Mode:** FOC (Field Oriented Control)

Connector Pinout

The motor uses an **XT30PB(2+2)** connector with 4 pins:

1. **Pin 1:** Power Supply Positive (+) - 48VDC
 2. **Pin 2:** Power Supply Negative (-) - GND
 3. **Pin 3:** CAN_L (CAN Low)
 4. **Pin 4:** CAN_H (CAN High)
-

Hardware Requirements

1. USB-to-CAN Adapter

You need a USB-to-CAN adapter compatible with Linux SocketCAN. Recommended options:

Budget Option:

- **CANable 2.0** (~\$30-40)
 - Open-source design
 - Native SocketCAN support with candlelight firmware
 - CAN-FD support
 - USB-C connector
 - Works out-of-the-box with Linux

Professional Options:

- **PEAK PCAN-USB** (~\$150-200)

- Industrial grade
 - Built-in Linux drivers
 - Excellent stability
 - Galvanic isolation available (opto-decoupled version)
- **Kvaser Leaf Light v2** (~\$250-300)
 - Professional-grade
 - Native Linux support
 - High reliability

2. CAN Wiring

- **CAN Cable:** Twisted pair cable for CAN_H and CAN_L
- **Termination:** 120Ω resistor at each end of the CAN bus
- **Power Supply:** 48V DC power supply (rated for at least 7A peak current)

3. Wiring Diagram

```
[48V PSU]
|
+--- [+48V] -----> Motor Pin 1 (Power +)
+--- [GND]  -----> Motor Pin 2 (Power -)

[USB-CAN Adapter]
|
+--- [CAN_H] ----> Motor Pin 4 (CAN_H)
+--- [CAN_L] ----> Motor Pin 3 (CAN_L)
+--- [120Ω] ----> Between CAN_H and CAN_L (termination)
```

Linux System Setup

Step 1: Install Required Packages

```
# Update system
sudo apt update
sudo apt upgrade

# Install CAN utilities
sudo apt install can-utils

# Install Python CAN library (optional, for Python control)
sudo apt install python3-pip
pip3 install python-can

# Install development tools (for C programming)
sudo apt install build-essential
```

Step 2: Load CAN Kernel Modules

```
# Load SocketCAN modules
sudo modprobe can
sudo modprobe can_raw
sudo modprobe slcan    # For serial line CAN adapters like CANable
```

To load automatically at boot, add to `/etc/modules`:

```
echo "can" | sudo tee -a /etc/modules
echo "can_raw" | sudo tee -a /etc/modules
```

Step 3: Configure CAN Interface

For Native SocketCAN Devices (PEAK, Kvaser with drivers)

```
# Set up CAN interface with 1 Mbps bitrate
sudo ip link set can0 type can bitrate 1000000

# Bring interface up
sudo ip link set up can0

# Verify interface is up
ip -details link show can0
```

For CANable with slcan Firmware

```
# Find the serial port (usually /dev/ttyACM0 or /dev/ttyUSB0)
ls /dev/tty*

# Attach slcan interface
sudo slcand -o -c -s8 /dev/ttyACM0 can0

# Bring interface up
sudo ip link set up can0
```

Note: `-s8` sets 1 Mbps baud rate for slcan

For CANable with candlelight Firmware

The CANable with candlelight firmware appears as a native CAN device:

```
sudo ip link set can0 type can bitrate 1000000
sudo ip link set up can0
```

Step 4: Verify CAN Interface

```
# Check interface status
ifconfig can0

# Expected output:
# can0: flags=193<UP,RUNNING,NOARP> mtu 16
#      unspec 00-00-00-00-00-00-00-00-00-00-00-00-00-00-00-00 txqueuelen 10  (UNSPEC)
```

Step 5: Create Startup Script (Optional)

Create `/usr/local/bin/can-setup.sh`:

```
#!/bin/bash
sudo ip link set can0 type can bitrate 1000000
sudo ip link set up can0
echo "CAN interface can0 is ready"
```

Make it executable:

```
sudo chmod +x /usr/local/bin/can-setup.sh
```

CAN Protocol Overview

Frame Format

The motor uses **CAN 2.0 Extended Frame** format (29-bit ID):

29-bit CAN ID Structure:

Bit 28-24: Communication Type (5 bits)
Bit 23-8: Data Area 2 (16 bits) - varies by command
Bit 7-0: Target Motor CAN_ID (8 bits)

Data Field: 8 bytes (Byte0 to Byte7)

Default Motor Settings

- **Default CAN_ID:** 0x01 (can be changed via command)
- **Master/Host CAN_ID:** 0xFD (recommended)
- **Baud Rate:** 1 Mbps (1000000 bps)

Key Communication Types

Type	Name	Description
0x0	Get Device ID	Request motor ID and MCU identifier
0x1	Operation Control Mode	Control with 5 parameters (angle, speed, Kp, Kd, torque)
0x2	Motor Feedback	Motor response with status, angle, speed, torque, temp
0x3	Motor Enable	Enable motor operation
0x4	Motor Stop	Stop motor and disable
0x6	Set Zero Position	Set current position as mechanical zero
0x7	Set CAN_ID	Change motor CAN_ID
0x11	Single Parameter Read	Read parameter by index
0x12	Single Parameter Write	Write parameter by index (volatile)
0x16	Save Parameters	Save parameters to non-volatile memory
0x18	Active Reporting	Enable/disable periodic status reporting

Control Modes

The motor supports 5 control modes, selected by writing to parameter `run_mode` (index 0x7005):

1. Operation Control Mode (run_mode = 0)

Description: Direct control with 5 parameters in one command **Parameters:**

- Target Angle: -4π to 4π rad
- Target Angular Velocity: -44 to 44 rad/s
- Kp: 0 to 500
- Kd: 0 to 5
- Torque: -17 to 17 N.m

Use Case: Real-time control with impedance (Kp/Kd gains)

2. Velocity Mode (run_mode = 2)

Description: Speed control **Parameters:**

- Speed Reference (spd_ref, 0x700A): -44 to 44 rad/s
- Current Limit (limit_cur, 0x7018): 0 to 23 A

Use Case: Constant speed applications

3. Current Mode (run_mode = 4)

Description: Direct current (Iq) control **Parameters:**

- Iq Reference (iq_ref, 0x7006): -23 to 23 A

Use Case: Torque control applications

4. Position Mode CSP (run_mode = 5)

Description: Cyclic Synchronous Position - continuous position updates **Parameters:**

- Position Reference (loc_ref, 0x7016): rad
- Speed Limit (limit_spd, 0x7017): 0 to 44 rad/s
- Current Limit (limit_cur, 0x7018): 0 to 23 A

Use Case: Real-time position tracking

5. Position Mode PP (run_mode = 1)

Description: Profile Position - motion with acceleration profile **Parameters:**

- Position Reference (loc_ref, 0x7016): rad
- Max Velocity (vel_max, 0x7024): rad/s
- Acceleration (acc_set, 0x7025): rad/s²

Use Case: Point-to-point motion with smooth acceleration

Command-Line Control

Basic Commands with can-utils

1. Monitor All CAN Traffic

```
candump can0
```

2. Enable Motor (Communication Type 3)

```
# CAN ID structure: 0x03 (type) + 00FD (host) + 01 (motor ID)
# Extended frame format
cansend can0 0300FD01#0000000000000000
```

Breakdown:

- 0300FD01: Extended CAN ID
 - 03: Communication Type 3 (Enable)
 - 00FD: Host CAN_ID (0xFD)
 - 01: Motor CAN_ID (0x01)
- #: Separator
- 0000000000000000: 8 data bytes (all zeros for enable command)

3. Stop Motor (Communication Type 4)

```
cansend can0 0400FD01#0000000000000000
```

4. Read Motor Feedback

After sending enable or control command, monitor feedback:

```
candump can0
# You'll see Type 2 response frames with motor status
```

5. Set Motor to Velocity Mode

First, write run_mode parameter (0x7005) to value 2:

```
# Type 18 (0x12), write parameter 0x7005 = 2
# CAN ID: 1200FD01 (Type 18, Host FD, Motor 01)
# Data: 05 70 00 00 02 00 00 00
#      ^^^^ index 0x7005 (little-endian: 05 70)
#      ^^^^ value 2 (uint8, little-endian: 02 00 00 00)
cansend can0 1200FD01#0570000002000000
```

Then save to memory (Type 22):

```
cansend can0 1600FD01#0102030405060708
```

6. Set Speed in Velocity Mode

Write speed to spd_ref (0x700A):

```
# Set speed to 5.0 rad/s
# Float 5.0 in hex (IEEE 754 little-endian): 00 00 A0 40

# CAN ID: 1200FD01
# Data: 0A 70 00 00 00 00 A0 40
cansend can0 1200FD01#0A70000000 00A040
```

7. Set Position in CSP Mode

First set mode to CSP (run_mode = 5):

```
cansend can0 1200FD01#0570000005000000
cansend can0 1600FD01#0102030405060708 # Save
```

Then set position (e.g., 3.14 rad):

```
# Float 3.14 in hex: C3 F5 48 40
cansend can0 1200FD01#1670000000C3F54840
```

Helper Script for Hex Conversion

Create float_to_hex.py:

```
#!/usr/bin/env python3
import struct
import sys

if len(sys.argv) != 2:
    print("Usage: ./float_to_hex.py <float_value>")
    sys.exit(1)

value = float(sys.argv[1])
hex_bytes = struct.pack('<f', value).hex()
print(f"Float {value} = 0x{hex_bytes} (little-endian)")
print(f"For cansend: {' '.join([hex_bytes[i:i+2] for i in range(0, 8, 2)])}")
```

Usage:

```
chmod +x float_to_hex.py
./float_to_hex.py 5.0
# Output: Float 5.0 = 0x0000a040 (little-endian)
#         For cansend: 00 00 a0 40
```

C Programming Examples

Example 1: Basic CAN Communication

File: can_motor_control.c

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <net/if.h>
#include <sys/socket.h>
#include <sys/ioctl.h>
#include <linux/can.h>
#include <linux/can/raw.h>

// Motor parameters
#define MOTOR_CAN_ID 0x01
#define HOST_CAN_ID 0xFD
#define CAN_INTERFACE "can0"

// Communication types
#define CMD_GET_ID 0x00
#define CMD_CONTROL 0x01
#define CMD_FEEDBACK 0x02
#define CMD_ENABLE 0x03
#define CMD_STOP 0x04
#define CMD_SET_ZERO 0x06
#define CMD_READ_PARAM 0x11
#define CMD_WRITE_PARAM 0x12
#define CMD_SAVE_PARAM 0x16

// Parameter ranges
#define P_MIN -12.57f
#define P_MAX 12.57f
#define V_MIN -44.0f
#define V_MAX 44.0f
#define T_MIN -17.0f
#define T_MAX 17.0f
#define KP_MIN 0.0f
#define KP_MAX 500.0f
#define KD_MIN 0.0f
#define KD_MAX 5.0f

// Global socket
int can_socket;

// Function to create extended CAN ID
uint32_t create_extended_id(uint8_t cmd_type, uint16_t data_area2, uint8_t motor_id) {
    uint32_t id = 0;
    id |= ((uint32_t)cmd_type & 0x1F) << 24; // Bits 28-24
    id |= ((uint32_t)data_area2 & 0xFFFF) << 8; // Bits 23-8
    id |= (motor_id & 0xFF); // Bits 7-0
    return id | CAN_EFF_FLAG; // Set extended frame flag
}

// Convert float to uint with range
uint16_t float_to_uint(float x, float x_min, float x_max, int bits) {
    float span = x_max - x_min;
    if (x > x_max) x = x_max;
    else if (x < x_min) x = x_min;
    return (uint16_t)((x - x_min) * ((float)((1 << bits) - 1)) / span);
}

// Initialize CAN socket
int can_init(const char *interface) {
    struct sockaddr_can addr;
    struct ifreq ifr;

    // Create socket
    can_socket = socket(PF_CAN, SOCK_RAW, CAN_RAW);

```

```

    if (can_socket < 0) {
        perror("Socket creation failed");
        return -1;
    }

    // Get interface index
    strcpy(ifr.ifr_name, interface);
    if (ioctl(can_socket, SIOCGIFINDEX, &ifr) < 0) {
        perror("ioctl SIOCGIFINDEX failed");
        close(can_socket);
        return -1;
    }

    // Bind socket to CAN interface
    memset(&addr, 0, sizeof(addr));
    addr.can_family = AF_CAN;
    addr.can_ifindex = ifr.ifr_ifindex;

    if (bind(can_socket, (struct sockaddr *)&addr, sizeof(addr)) < 0) {
        perror("Socket bind failed");
        close(can_socket);
        return -1;
    }

    printf("CAN socket initialized on %s\n", interface);
    return 0;
}

// Send CAN frame
int can_send_frame(uint32_t can_id, uint8_t *data, uint8_t len) {
    struct can_frame frame;

    frame.can_id = can_id;
    frame.can_dlc = len;
    memcpy(frame.data, data, len);

    int nbytes = write(can_socket, &frame, sizeof(frame));
    if (nbytes != sizeof(frame)) {
        perror("CAN write failed");
        return -1;
    }

    return 0;
}

// Receive CAN frame with timeout
int can_receive_frame(struct can_frame *frame, int timeout_ms) {
    fd_set readfds;
    struct timeval timeout;

    FD_ZERO(&readfds);
    FD_SET(can_socket, &readfds);

    timeout.tv_sec = timeout_ms / 1000;
    timeout.tv_usec = (timeout_ms % 1000) * 1000;

    int ret = select(can_socket + 1, &readfds, NULL, NULL, &timeout);
    if (ret < 0) {
        perror("Select failed");
        return -1;
    } else if (ret == 0) {
        return 0; // Timeout
    }

    int nbytes = read(can_socket, frame, sizeof(*frame));
    if (nbytes < 0) {

```



```

        perror("CAN read failed");
        return -1;
    }

    return 1; // Frame received
}

// Enable motor
int motor_enable(uint8_t motor_id) {
    uint32_t can_id = create_extended_id(CMD_ENABLE, HOST_CAN_ID, motor_id);
    uint8_t data[8] = {0};

    printf("Enabling motor ID %d...\n", motor_id);
    return can_send_frame(can_id, data, 8);
}

// Stop motor
int motor_stop(uint8_t motor_id) {
    uint32_t can_id = create_extended_id(CMD_STOP, HOST_CAN_ID, motor_id);
    uint8_t data[8] = {0};

    printf("Stopping motor ID %d...\n", motor_id);
    return can_send_frame(can_id, data, 8);
}

// Write parameter (float)
int motor_write_param_float(uint8_t motor_id, uint16_t index, float value) {
    uint32_t can_id = create_extended_id(CMD_WRITE_PARAM, HOST_CAN_ID, motor_id);
    uint8_t data[8] = {0};

    // Index (little-endian)
    data[0] = index & 0xFF;
    data[1] = (index >> 8) & 0xFF;
    data[2] = 0;
    data[3] = 0;

    // Value (float, little-endian)
    memcpy(&data[4], &value, sizeof(float));

    return can_send_frame(can_id, data, 8);
}

// Write parameter (uint8)
int motor_write_param_uint8(uint8_t motor_id, uint16_t index, uint8_t value) {
    uint32_t can_id = create_extended_id(CMD_WRITE_PARAM, HOST_CAN_ID, motor_id);
    uint8_t data[8] = {0};

    data[0] = index & 0xFF;
    data[1] = (index >> 8) & 0xFF;
    data[2] = 0;
    data[3] = 0;
    data[4] = value;

    return can_send_frame(can_id, data, 8);
}

// Save parameters to flash
int motor_save_params(uint8_t motor_id) {
    uint32_t can_id = create_extended_id(CMD_SAVE_PARAM, HOST_CAN_ID, motor_id);
    uint8_t data[8] = {1, 2, 3, 4, 5, 6, 7, 8};

    printf("Saving parameters for motor ID %d...\n", motor_id);
    return can_send_frame(can_id, data, 8);
}

// Set control mode

```

```

int motor_set_mode(uint8_t motor_id, uint8_t mode) {
    printf("Setting motor to mode %d...\n", mode);
    if (motor_write_param_uint8(motor_id, 0x7005, mode) < 0) {
        return -1;
    }
    usleep(10000); // 10ms delay
    return motor_save_params(motor_id);
}

// Control in operation mode (Type 1)
int motor_control_operation(uint8_t motor_id, float position, float velocity,
                           float kp, float kd, float torque) {
    uint32_t can_id = create_extended_id(CMD_CONTROL, (uint16_t)(float_to_uint(torque, T_MIN, T_MAX, 16)) << 8 | HOST_CAN_ID, motor_id);
    uint8_t data[8];

    uint16_t pos_uint = float_to_uint(position, P_MIN, P_MAX, 16);
    uint16_t vel_uint = float_to_uint(velocity, V_MIN, V_MAX, 16);
    uint16_t kp_uint = float_to_uint(kp, KP_MIN, KP_MAX, 16);
    uint16_t kd_uint = float_to_uint(kd, KD_MIN, KD_MAX, 16);

    data[0] = (pos_uint >> 8) & 0xFF;
    data[1] = pos_uint & 0xFF;
    data[2] = (vel_uint >> 8) & 0xFF;
    data[3] = vel_uint & 0xFF;
    data[4] = (kp_uint >> 8) & 0xFF;
    data[5] = kp_uint & 0xFF;
    data[6] = (kd_uint >> 8) & 0xFF;
    data[7] = kd_uint & 0xFF;

    return can_send_frame(can_id, data, 8);
}

// Set velocity
int motor_set_velocity(uint8_t motor_id, float velocity_rad_s) {
    return motor_write_param_float(motor_id, 0x700A, velocity_rad_s);
}

// Set position
int motor_set_position(uint8_t motor_id, float position_rad) {
    return motor_write_param_float(motor_id, 0x7016, position_rad);
}

// Main example
int main(int argc, char *argv[]) {
    // Initialize CAN
    if (can_init(CAN_INTERFACE) < 0) {
        return 1;
    }

    printf("Motor Control Example\n");
    printf("=====\n");

    // Enable motor
    if (motor_enable(MOTOR_CAN_ID) < 0) {
        goto cleanup;
    }
    sleep(1);

    // Set to velocity mode
    printf("\nSetting velocity mode...\n");
    if (motor_set_mode(MOTOR_CAN_ID, 2) < 0) { // Mode 2 = Velocity
        goto cleanup;
    }
    sleep(1);

    // Run at 5 rad/s for 3 seconds

```

```
printf("\nRunning at 5 rad/s...\n");
if (motor_set_velocity(MOTOR_CAN_ID, 5.0) < 0) {
    goto cleanup;
}
sleep(3);

// Stop
printf("\nStopping motor...\n");
motor_set_velocity(MOTOR_CAN_ID, 0.0);
sleep(1);

motor_stop(MOTOR_CAN_ID);

cleanup:
    close(can_socket);
    return 0;
}
```

Compile and Run

```
gcc can_motor_control.c -o can_motor_control
sudo ./can_motor_control
```

Python Programming Examples

Example 1: Basic Motor Control

File: `motor_control.py`

```

#!/usr/bin/env python3
import can
import struct
import time

# Configuration
CAN_INTERFACE = 'can0'
MOTOR_ID = 0x01
HOST_ID = 0xFD

# Communication types
CMD_ENABLE = 0x03
CMD_STOP = 0x04
CMD_WRITE_PARAM = 0x12
CMD_SAVE_PARAM = 0x16
CMD_CONTROL = 0x01

# Parameter indices
PARAM_RUN_MODE = 0x7005
PARAM_SPD_REF = 0x700A
PARAM_LOC_REF = 0x7016
PARAM_IQ_REF = 0x7006

# Initialize CAN bus
bus = can.interface.Bus(channel=CAN_INTERFACE, bustype='socketcan')

def create_extended_id(cmd_type, data_area2, motor_id):
    """Create 29-bit extended CAN ID"""
    can_id = 0
    can_id |= (cmd_type & 0x1F) << 24
    can_id |= (data_area2 & 0xFFFF) << 8
    can_id |= (motor_id & 0xFF)
    return can_id

def motor_enable(motor_id=MOTOR_ID):
    """Enable motor"""
    can_id = create_extended_id(CMD_ENABLE, HOST_ID, motor_id)
    msg = can.Message(
        arbitration_id=can_id,
        data=[0, 0, 0, 0, 0, 0, 0, 0],
        is_extended_id=True
    )
    bus.send(msg)
    print(f"Motor {motor_id} enabled")

def motor_stop(motor_id=MOTOR_ID):
    """Stop motor"""
    can_id = create_extended_id(CMD_STOP, HOST_ID, motor_id)
    msg = can.Message(
        arbitration_id=can_id,
        data=[0, 0, 0, 0, 0, 0, 0, 0],
        is_extended_id=True
    )
    bus.send(msg)
    print(f"Motor {motor_id} stopped")

def write_param_float(motor_id, param_index, value):
    """Write float parameter"""
    can_id = create_extended_id(CMD_WRITE_PARAM, HOST_ID, motor_id)

    # Pack data
    data = bytearray(8)
    data[0] = param_index & 0xFF
    data[1] = (param_index >> 8) & 0xFF
    data[2] = 0

```

```

data[3] = 0
# Float in little-endian
data[4:8] = struct.pack('<f', value)

msg = can.Message(
    arbitration_id=can_id,
    data=data,
    is_extended_id=True
)
bus.send(msg)

def write_param_uint8(motor_id, param_index, value):
    """Write uint8 parameter"""
    can_id = create_extended_id(CMD_WRITE_PARAM, HOST_ID, motor_id)

    data = bytearray(8)
    data[0] = param_index & 0xFF
    data[1] = (param_index >> 8) & 0xFF
    data[2] = 0
    data[3] = 0
    data[4] = value & 0xFF

    msg = can.Message(
        arbitration_id=can_id,
        data=data,
        is_extended_id=True
    )
    bus.send(msg)

def save_params(motor_id=MOTOR_ID):
    """Save parameters to flash"""
    can_id = create_extended_id(CMD_SAVE_PARAM, HOST_ID, motor_id)
    msg = can.Message(
        arbitration_id=can_id,
        data=[1, 2, 3, 4, 5, 6, 7, 8],
        is_extended_id=True
    )
    bus.send(msg)
    print(f"Parameters saved for motor {motor_id}")

def set_control_mode(motor_id, mode):
    """
    Set control mode:
    0 = Operation Control Mode
    1 = Position Mode (PP)
    2 = Velocity Mode
    4 = Current Mode
    5 = Position Mode (CSP)
    """
    print(f"Setting control mode to {mode}...")
    write_param_uint8(motor_id, PARAM_RUN_MODE, mode)
    time.sleep(0.01)
    save_params(motor_id)
    time.sleep(0.1)

def set_velocity(motor_id, velocity_rad_s):
    """Set velocity in rad/s (for Velocity Mode)"""
    write_param_float(motor_id, PARAM_SPD_REF, velocity_rad_s)
    print(f"Velocity set to {velocity_rad_s} rad/s")

def set_position(motor_id, position_rad):
    """Set position in radians (for CSP Mode)"""
    write_param_float(motor_id, PARAM_LOC_REF, position_rad)
    print(f"Position set to {position_rad} rad")

def set_current(motor_id, current_a):

```

```

    """Set current in Amps (for Current Mode)"""
    write_param_float(motor_id, PARAM_IQ_REF, current_a)
    print(f"Current set to {current_a} A")

def read_feedback():
    """Read motor feedback (Type 2 messages)"""
    msg = bus.recv(timeout=1.0)
    if msg:
        # Extract communication type from extended ID
        cmd_type = (msg.arbitration_id >> 24) & 0x1F
        if cmd_type == 0x02: # Feedback message
            # Parse data
            angle_raw = (msg.data[0] << 8) | msg.data[1]
            vel_raw = (msg.data[2] << 8) | msg.data[3]
            torque_raw = (msg.data[4] << 8) | msg.data[5]
            temp_raw = (msg.data[6] << 8) | msg.data[7]

            # Convert to physical units
            angle = (angle_raw / 65535.0) * (8 * 3.14159) - (4 * 3.14159)
            velocity = (vel_raw / 65535.0) * 88 - 44
            torque = (torque_raw / 65535.0) * 34 - 17
            temperature = temp_raw / 10.0

            print(f"Angle: {angle:.2f} rad, Vel: {velocity:.2f} rad/s, "
                  f"Torque: {torque:.2f} Nm, Temp: {temperature:.1f}°C")
            return angle, velocity, torque, temperature
        return None

# Example usage
if __name__ == "__main__":
    try:
        print("Fulling Motor Control Example")
        print("=" * 50)

        # Enable motor
        motor_enable()
        time.sleep(1)

        # Example 1: Velocity control
        print("\n--- Velocity Control Test ---")
        set_control_mode(MOTOR_ID, 2) # Velocity mode
        time.sleep(0.5)

        set_velocity(MOTOR_ID, 5.0) # 5 rad/s
        print("Running for 3 seconds...")
        for i in range(3):
            time.sleep(1)
            read_feedback()

        set_velocity(MOTOR_ID, 0.0)
        time.sleep(1)

        # Example 2: Position control
        print("\n--- Position Control Test ---")
        set_control_mode(MOTOR_ID, 5) # CSP mode
        time.sleep(0.5)

        positions = [0, 1.57, 3.14, 0] # 0, 90°, 180°, 0
        for pos in positions:
            print(f"\nMoving to {pos:.2f} rad...")
            set_position(MOTOR_ID, pos)
            time.sleep(2)
            read_feedback()

        # Stop motor
        print("\n--- Stopping Motor ---")

```

```
        motor_stop()

except KeyboardInterrupt:
    print("\nInterrupted by user")
    motor_stop()
finally:
    bus.shutdown()
```

Run Python Script

```
chmod +x motor_control.py
sudo python3 motor_control.py
```

Troubleshooting

Problem 1: CAN Interface Not Found

Symptoms: `ifconfig` doesn't show `can0`

Solutions:

1. Check USB adapter is connected: `lsusb`
2. Load kernel modules:

```
sudo modprobe can
sudo modprobe can_raw
```

3. For `slcan` devices, attach manually:

```
sudo slcand -o -c -s8 /dev/ttyACM0 can0
```

Problem 2: Motor Not Responding

Symptoms: Commands sent but no feedback received

Check:

1. **Power:** Verify 48V power supply is connected
2. **Wiring:** Check CAN_H and CAN_L connections
3. **Termination:** Ensure 120Ω resistor between CAN_H and CAN_L
4. **Baud rate:** Must be 1 Mbps (1000000)
5. **Motor ID:** Default is 0x01, verify with `candump can0`

Problem 3: Permission Denied

Symptoms: Operation not permitted when using CAN commands

Solutions:

1. Run with `sudo`: `sudo cansend can0 ...`
2. Add user to `dialout` group:

```
sudo usermod -aG dialout $USER
# Log out and log back in
```

Problem 4: CAN Bus Errors

Symptoms: Error frames in `candump`

Check:

1. **Bus-off state:** Reset interface

```
sudo ip link set can0 down
sudo ip link set can0 up
```

- 2. **Cable quality:** Use twisted pair cable
- 3. **Cable length:** Keep under 40m for 1 Mbps
- 4. **Termination:** Must have exactly two 120Ω resistors (one at each end)

Problem 5: Motor Overheating

Symptoms: Motor stops, temperature >135°C

Solutions:

- 1. Reduce current limit: `limit_cur` parameter (0x7018)
- 2. Reduce torque limit: `limit_torque` parameter (0x7007)
- 3. Check for mechanical binding
- 4. Allow cooling time between operations

Reference Tables

Important Parameter Indices

Index	Name	Type	R/W	Range	Default	Description
0x7005	run_mode	uint8	R/W	0-5	-	Control mode selection
0x7006	iq_ref	float	R/W	-23 to 23 A	-	Current mode Iq command
0x700A	spd_ref	float	R/W	-44 to 44 rad/s	-	Velocity mode speed command
0x7016	loc_ref	float	R/W	rad	-	Position mode angle command
0x7017	limit_spd	float	R/W	0 to 44 rad/s	-	CSP mode speed limit
0x7018	limit_cur	float	R/W	0 to 23 A	23	Velocity/position mode current limit
0x7019	mechPos	float	R	rad	-	Mechanical angle of load
0x701A	iqf	float	R	-16 to 16 A	-	iq filter value
0x701B	mechVel	float	R	-44 to 44 rad/s	-	Speed of load
0x701C	VBUS	float	R	V	-	Bus voltage
0x701E	loc_kp	float	R/W	-	30	Position mode Kp
0x701F	spd_kp	float	R/W	-	2	Speed mode Kp
0x7020	spd_ki	float	R/W	-	0.021	Speed mode Ki
0x7024	vel_max	float	R/W	rad/s	10	PP mode max velocity
0x7025	acc_set	float	R/W	rad/s²	10	PP mode acceleration
0x7028	canTimeout	uint32	R/W	-	0	CAN timeout (20000 = 1s, 0 = disabled)

Control Mode Summary

Mode Value	Mode Name	Description	Key Parameters
0	Operation Control	Real-time control with Kp/Kd	Use Type 1 command
1	Position PP	Profile position with acceleration	loc_ref, vel_max, acc_set
2	Velocity	Speed control	spd_ref, limit_cur
4	Current	Direct Iq current control	iq_ref
5	Position CSP	Cyclic synchronous position	loc_ref, limit_spd, limit_cur

CAN Command Quick Reference

Command	CAN ID Example	Data	Description
Enable	0300FD01	00 00 00 00 00 00 00 00	Enable motor
Stop	0400FD01	00 00 00 00 00 00 00 00	Stop motor
Write Param	1200FD01	IDX_L IDX_H 00 00 VAL[0-3]	Write parameter
Save Params	1600FD01	01 02 03 04 05 06 07 08	Save to flash
Set Mode	1200FD01	05 70 00 00 MODE 00 00 00	Set run_mode

Conversion Formulas

Angle (16-bit):

```
Angle (rad) = (raw_value / 65535) * 8π - 4π
raw_value = ((angle + 4π) / 8π) * 65535
```


Velocity (16-bit):

```
Velocity (rad/s) = (raw_value / 65535) * 88 - 44  
raw_value = ((velocity + 44) / 88) * 65535
```

Torque (16-bit):

```
Torque (Nm) = (raw_value / 65535) * 34 - 17  
raw_value = ((torque + 17) / 34) * 65535
```

Float (IEEE 754 little-endian):

```
import struct  
  
# Float to bytes  
bytes_data = struct.pack('<f', float_value)  
  
# Bytes to float  
float_value = struct.unpack('<f', bytes_data)[0]
```

Safety Notes

1. **Power:** Always connect power before CAN communication
2. **Emergency Stop:** Keep motor stop command ready (0400FD01#00...)
3. **Current Limits:** Set appropriate current limits to prevent overheating
4. **Testing:** Test with low speeds/torques first
5. **Monitoring:** Monitor temperature (should stay < 135°C)
6. **Mechanical:** Ensure motor shaft is free to move before enabling

Additional Resources

- **Fulling Motor Website:** <https://fullingmotor.com> (<https://fullingmotor.com>)
- **SocketCAN Documentation:** <https://www.kernel.org/doc/Documentation/networking/can.txt> (<https://www.kernel.org/doc/Documentation/networking/can.txt>)
- **can-utils GitHub:** <https://github.com/linux-can/can-utils> (<https://github.com/linux-can/can-utils>)
- **python-can Documentation:** <https://python-can.readthedocs.io> (<https://python-can.readthedocs.io>)

Document Version: 1.0

Last Updated: October 2025

Motor Model: iSVD78C-48V-230-A14-002